

A boundary element-based automatic domain partitioning approach for semi-structured quad mesh generation

Zhoufang Xiao^{a,b}, Shouping He^a, Gang Xu^{a,*}, Jianjun Chen^c, Qing Wu^a

^a College of Computer Science and Technology, Hangzhou Dianzi University, Hangzhou 310018, China

^b Advanced Numerical Simulation Laboratory, University of British Columbia, Vancouver, BC, V6T 1Z4, Canada

^c Center for Engineering and Scientific Computation, Zhejiang University, Hangzhou 310027, China

ARTICLE INFO

Keywords:

Domain partitioning
Quad mesh generation
Semi-structured mesh
Boundary element method
Cross-field

ABSTRACT

In this paper, a boundary element-based approach is proposed for partitioning a planar domain automatically into a set of 4-sided regions, which is suitable for semi-structured quad mesh generation. The basic idea is to generate a smooth cross-field via solving PDEs with the boundary element method and then partition the input domain by extracting singular structures of the cross-field. Firstly, a cross is configured at each boundary vertex of the input domain. Then, two Laplacian equations are selected as the governing equation to smoothly propagate the crosses defined on the boundary into the interior of the domain. To obtain an accurate cross-field, the boundary element method is used to solve the governing equations. Then the singular vertices are identified by analyzing the structure of the cross-field, and the streamlines emanating from these points are traced and simplified as segmentation curves to partition the domain. Finally, to demonstrate the efficiency and effectiveness of the proposed approach, some quad mesh generation examples and comparison with previous approaches are presented.

1. Introduction

Computational fluid dynamics (CFD) has become an indispensable tool in various scientific and engineering communities. A prominent issue of applying CFD in real applications is how to create a high-quality mesh to discretize the computational domain automatically. Mesh generation usually takes a major portion of wall-clock time during a CFD simulation, and the solution speed and accuracy depend highly on the quality of the meshes used by the solver. For 2D problems, a quadrilateral mesh normally has higher solution accuracy and lower degrees of freedom than a triangular mesh, thus it is preferred for numerical simulation.

Nevertheless, it remains a challenging task to create high-quality quadrilateral meshes automatically [1,2]. Currently, the main types of quadrilateral mesh generation methods are geometric heuristic based methods [3,4], parametrization based methods [5–7], cross-field based methods [8–11], and Morse theory based methods [12–14]. A comprehensive review of all these methods can be found in the recent survey by Bommes [5] and Fang [1]. Many of these methods generate quadrilateral meshes, structured or unstructured, directly on the entire domain. For complex geometries, lower quality or even degenerated elements may be obtained using such methods [15]. An alternate way is to firstly partition the domain into sub-regions which are often much simpler than

the original domain, and then generate quadrilateral meshes for each sub-region individually.

The medial axis is often used to decompose a domain [15]. However, the medial axis is numerically unstable; a small change of the domain can affect the decomposition result heavily. Recently, Kowalski [11] proposed a cross-field based domain partitioning method for planar surface. In this method, a normalized vector-field was obtained by solving a constrained heat transfer problem and a smooth cross-field was finally obtained and used to partition the domain. Each generated sub-domain was four-sided and could be easily meshed using a structured method. To obtain a similar domain partitioning result, Fogg [10] proposed a method to generate smooth cross-fields on triangulated surfaces from an arbitrary number of alignment constraints. Fast-marching algorithm was then used to propagate the field into regions between constraints. A strong point of this method is that the generated cross-field can abide to a prescribed sizing field by solving an energy function in the propagating step.

In this paper, a new domain partitioning method for semi-structured quadrilateral mesh generation is proposed, which is mainly based on analysing the cross-field obtained by solving two Laplacian equations with the boundary element method. More specifically, with the one-to-one mapping between vectors and crosses defined in 2D [11,16], crosses defined on the boundary of the domain are used to generate a smooth

* Corresponding author.

E-mail address: gxu@hdu.edu.cn (G. Xu).

cross-field in the interior region by propagating the corresponding vectors on the boundary into the interior region. This propagation of vectors is governed by two Laplacian equations. Mathematically speaking, the solution of these Laplacian equations could smoothly propagate the vectors defined on boundary into the domain interior, thus leading to a smooth vector-field and a smooth cross-field. Then, a similar method presented in [11] was used to partition the domain, identifying singular points from the vector-field and computing streamlines emanating from these points as lines to partition the problem domain into a set of meshable subdomains. In our experiments, it was observed that the domain partitioning results created by the proposed approach had better singular topology structures than those created by other approaches based on parameterization and Morse theory [1].

Because the Laplacian equation is used as the governing equation, we can now use the boundary-element method (BEM for short) [17–20] to numerically compute the vector-field. Besides, the BEM only needs the boundary segments as input. However, numerical schemes based on a background mesh need an interpolation procedure, and could thus hardly achieve the same level of accuracy as the BEM, unless very fine background elements are placed in the interior region. Based on an accurate cross-field, streamlines can be computed with less numerical issues. Besides, the fast multipole method can be used to solve boundary integral equations in the BEM, which helps to substantially reduce the solution time of cross-field [17,20]. Thus, the whole process of the proposed method could be more efficient than those based on solving optimization problems.

The contributions of this study can be summarized in three aspects: (1) Two Laplacian equations are used to govern the propagation of the crosses defined on the domain boundary into the interior region to generate a smooth cross-field. This behavior can also make the constraints set on the boundary be well satisfied in the generated cross-field; (2) BEM is used to solve the governing PDEs, which not only relaxes the requirements on the quality of the input background triangular mesh compared to other numerical methods but also increases the solution accuracy for the PDEs because of the use of boundary integrals; (3) A scheme is proposed to simplify the initial streamlines generated from the cross-field, which extends the proposed domain partitioning method to complex geometries.

The remainder of this paper is organized as follows. Section 2 gives an overview of the proposed framework. Section 3 describes the governing equation of the cross-field and its solution scheme. Section 4 introduces some important implementation details involved in this method, and then the mesh generation process of all these subdomains is demonstrated in Section 5. Section 6 presents various numerical experiments that demonstrate the effectiveness and efficiency of the proposed scheme. Section 7 concludes with the outcomes of the study and points to some directions for future studies.

2. Overview of the algorithm

The proposed method takes a 2D geometric model as the input and outputs a domain decomposition result automatically, and it mainly includes the following three steps (see Fig. 1):

1. Creating the background mesh. The domain boundary $\partial\Omega$ is divided into line segments which are used to discretize the boundary integral equation as shown in Eq. (2), while the interior region of the domain, Ω , is discretized with a triangular mesh used as the background mesh of the cross-field.
2. Solution of cross-field. Instead of computing the cross-field directly, a vector-field is first computed by solving two Laplacian equations subject to Neumann boundary conditions. Then, with the help of mapping between a vector and a cross defined in Section 2.1, the corresponding cross-field over the interior region of Ω is obtained.

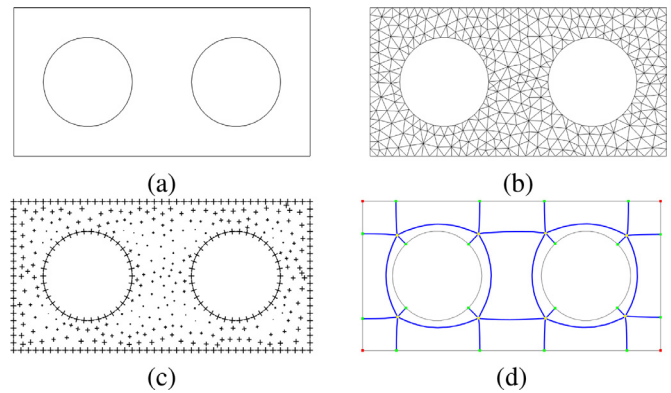


Fig. 1. The main steps in the proposed automatic domain partitioning method: (a) The input boundary geometry; (b) The background mesh; (c) The cross-field; (d) The domain decomposition results.

3. Domain decomposition. Firstly, the singular points of the cross-field are identified by analysing the vector-field obtained in the previous step. Note that, the L2-norm of the vector at any singular point is equal to 0. Then, for each singular point p , compute all the streamlines of the cross-field starting or ending at p . Such streamlines are used as lines to partition the domain into four-sided subdomains.

The triangular mesh in the first step can be generated with any qualified method, i.e., the Delaunay method is used in this study, which can discretize the interior region evenly. The triangular mesh here is not used to solve the PDEs numerically thus the mesh quality is not concerned, but a vector and a cross are calculated and stored at each mesh node to represent the piece-wise linear vector-field and cross-field over the interior region.

The second step is the main step in the proposed method. To generate a cross-field over a computational domain, a PDE-based method is used to smoothly propagate the cross defined on the boundary into the interior region. Following this idea, boundary aligned crosses could be set on the boundary and be propagated smoothly into far region. Besides, the crosses represent the final quadrilateral mesh to some extent [5]. Thus, the boundary aligned crosses near the boundary in turn lead to boundary aligned quadrilateral mesh which is preferred in numerical simulations.

With the generated cross-field as input, we finally identify the singular points and trace the streamlines emanating from them as the lines to partition the domain. The streamlines are traced with the Runge-Kutta method over the background triangular mesh [11], and according to the type of singular points, i.e., valence-3 points or valence-5 points, different numbers of streamlines will coincide at them. In this procedure, numerical errors would exist and some simplifications on the streamlines are needed in order to get the final decomposition result.

3. The governing equations for cross-field

The computation of cross-field is a crucial step in domain decomposition. In this study, a similar method as in [11] is proposed to generate the cross-field over the domain. It tries to propagate the crosses defined on the boundary of the domain into the interior region in a smooth way. Instead of propagating the crosses directly, a reference vector is defined for each cross and the corresponding reference vectors of crosses defined on the boundary are propagated into the interior region. Being different from the method in [11], the propagation problem of vectors is implemented by solving two Laplacian-type governing equations, and BEM is used to solve these equations to obtain accurate results. In this section, the mapping between a vector and a cross is reviewed first, and then the governing equations and its BEM solution scheme are demonstrated.

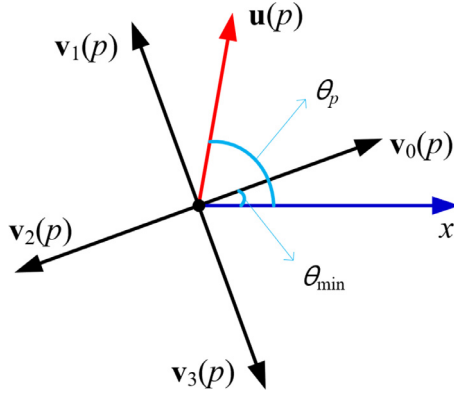


Fig. 2. The mapping between a vector and a cross.

3.1. The mapping between a vector and a cross

In 2D, a unique reference vector can be defined to represent a cross [11,16]. Let $\mathbf{v}_k(p)$ ($k = 0, 1, 2, 3$) be the cross at the point p , $\mathbf{u}(p)$ the corresponding reference vector, and $\theta_{c,k}(p)$ ($k = 0, 1, 2, 3$) and $\theta(p)$ the counterclockwise rotation angles from x axis to $\mathbf{v}_k(p)$ and $\mathbf{u}(p)$, respectively. Then the mapping between the reference vector and the cross at this point can be defined as the following (see Fig. 2):

$$\begin{cases} \|\mathbf{u}(p)\| = \|\mathbf{v}_k(p)\| & k = 0, 1, 2, 3 \\ \theta(p) = 4\theta \end{cases} \quad (1)$$

where, $\|\cdot\|$ stands for the L2-norm, and $\theta = \min\{\theta_{c,0}(p), \theta_{c,1}(p), \theta_{c,2}(p), \theta_{c,3}(p)\}$. With this definition of mapping, the cross only rotates one-quarter circle when the reference vector rotates a circle around the original point of the axis. As a cross is 4-way rotational symmetry [8,16], it remains the same when a cross rotates one-quarter circle. Note that unlike the notions used in [11,16] is that, both the vectors and crosses used in this study are not constrained to be unit. In fact, their L2-norms become smaller and smaller as they are propagated into the interior region of the domain.

3.2. The governing equations and their solution scheme

Denote the problem domain by Ω , and its boundary by $\partial\Omega$. Let $\mathbf{u}_0(p)$ be the vector defined on any boundary point p . Our goal is to propagate $\mathbf{u}_0(p)$ into the interior region of Ω smoothly. Similar to the approach we proposed in [21] for computing the marching directions in boundary layer mesh generation, two Laplacian equations, together with the boundary conditions, can be deduced to compute the vectors in the interior region of Ω :

$$\begin{cases} \nabla^2 u_i(p) = 0 & p \in \Omega \\ u_i(p) = u_{i,0}(p) & p \in \partial\Omega \end{cases} \quad (i = x, y). \quad (2)$$

Here, ∇^2 denotes the Laplacian operator ($= \partial^2/\partial x^2 + \partial^2/\partial y^2$), $u_i(p)$ denotes the corresponding component of the vector function in the interior region of Ω , and $u_{i,0}(p)$ is the corresponding component of $\mathbf{u}_0(p)$, defining the Dirichlet boundary condition.

Basically, the solution of $u_i(p)$ ($i = x, y$) at point p_0 could be computed by the boundary element method (BEM) with the following boundary integration equation [17]:

$$\varphi(p_0)u_i(p_0) = \int_{\partial\Omega} \left[G(p_0, p) \frac{\partial u_i(p)}{\partial \mathbf{n}} - u_i(p) \frac{\partial G(p_0, p)}{\partial \mathbf{n}} \right] ds_p. \quad (3)$$

Here, $p \in \partial\Omega$, ds_p represents the length of the small segment in the vicinity of p , and $G(p_0, p) = -\ln r/(2\pi)$ is the fundamental solution of the Laplacian equation, in which $r = |p_0 - p|$ is the distance between the source point p_0 and the field point p . The value of the coefficient $\varphi(p_0)$ depends on the position of p_0 , and it is described as below:

$$\varphi(p_0) = \begin{cases} \frac{1}{2}, & p_0 \in s(\partial\Omega) \\ 1, & p_0 \in i(\Omega) \end{cases}, \quad (4)$$

where $s(\partial\Omega)$ denotes the smooth boundary around p_0 , and $i(\Omega)$ denotes the interior region of Ω .

Note that, as seen in Eq. (2), for a point belonging to $\partial\Omega$, $u_i(p)$ is given while $\partial u_i(p)/\partial \mathbf{n}$ is unknown. Therefore, it needs to compute the value of $\partial u_i(p)/\partial \mathbf{n}$ first before computing the value of $u_i(p_0)$ ($p_0 \in i(\Omega)$). Fortunately, this can be done by using Eq. (3) when $p_0 \in s(\partial\Omega)$.

To solve for the unknown boundary variables numerically with BEM, boundary elements are used to discretize the boundary integral equation defined in Eq. (3). In this study, the constant elements are adopted. Suppose $\partial\Omega$ is divided into N elements (denoted as $\Delta\partial\Omega_k$ ($k = 1 \sim N$)), and one node is placed on each element (denoted as p_k ($k = 1 \sim N$)). Then Eq. (3) could be discretized into the following form:

$$\frac{1}{2}u_i(p_k) = \sum_{j=1}^N \int_{\Delta\partial\Omega_j} \left(G(p_k, p_j) \frac{\partial u_i(p_j)}{\partial \mathbf{n}} - \frac{\partial G(p_k, p_j)}{\partial \mathbf{n}} u_i(p_j) \right) ds_p. \quad (5)$$

There are totally N unknown variables in Eq. (5). For each point p_k ($k = 1 \sim N$), an equation in the form of Eq. (5) can be formulated. Therefore, a total of N linear equations can be used to compute the N unknowns.

Let $q_j = \frac{\partial u_i(p_j)}{\partial \mathbf{n}}$, $u_j = u_i(p_j)$, $g_{kj} = \int_{\Delta\partial\Omega_j} G(p_k, p_j) ds_p$, and $f_{kj} = \int_{\Delta\partial\Omega_j} \frac{\partial G(p_k, p_j)}{\partial \mathbf{n}} ds_p$, then, Eq. (5) for all points can be rewritten as,

$$\frac{1}{2}u_k = \sum_{j=1}^N [g_{kj}q_j - f_{kj}u_j], \quad k = 1, 2, \dots, N. \quad (6)$$

For a given j , only q_j or u_j is known. By rearranging the known and unknown variables in Eq. (6), we can get a linear system in matrix form as below:

$$\mathbf{A}\lambda = \mathbf{b}, \quad (7)$$

where $\mathbf{A}$ is the coefficient matrix, λ and $\mathbf{b}$ are the unknown and known vectors, respectively. In BEM, $\mathbf{A}$ and $\mathbf{b}$ need $O(n^3)$ and $O(n^2)$ operations to be computed, respectively. The unknown vector λ can be computed by solving Eq. (7) with a direct or iterative method.

Once all the unknown boundary variables are obtained, the discretized form of Eq. (3) can then be used to compute the solution of $u_i(p)$ at any arbitrary point p_0 located inside Ω :

$$u_i(p_0) = \sum_{j=1}^N \int_{\Delta\partial\Omega_j} \left(G(p_0, p_j) \frac{\partial u_i(p_j)}{\partial \mathbf{n}} - \frac{\partial G(p_0, p_j)}{\partial \mathbf{n}} u_i(p_j) \right) ds_p. \quad (8)$$

4. The automatic domain partitioning method

In the next two subsections, more implementation details of the last two steps of the proposed method will be given.

4.1. Solution of cross-field

Instead of computing the cross-field directly, we first compute a vector-field, and then use the one-to-one mapping between vectors and crosses to compute the cross-field. The vector-field over the problem domain is generated by propagating the vectors defined on the boundary into the interior region. Section 3.2 gives the governing equations for the propagation problem of vectors. In this section, the details of the implementation of the solution of the cross-field are demonstrated (see Algorithm 1).

Given a domain in 2D and its interior region discretized with a triangular mesh which is composed of m boundary segments, n triangular elements and l nodes denoted by

$$M_t = \{S = \{s_i | i = 1, 2, L, m\}, \\ T = \{t_i | i = 1, 2, L, n\}, P = \{p_i | i = 1, 2, L, l\}\},$$

Algorithm 1 Solution of cross-field

1. Input: $\mathbf{v}_k(p)$ ($k = 0, 1, 2, 3; p \in \partial\Omega$)
2. Compute $u_i(p)$ ($i = x, y; p \in \partial\Omega$) with one-to-one mapping between vectors and crosses
3. Compute boundary values ($\partial u_i(p)/\partial n$ ($i = x, y; p \in s_j$)) for each boundary element s_j with BEM
4. **#pragma omp parallel num_threads(M_1)**
5. Iterate on all nodes
6. Compute the normal vector $u_i(p)$ ($i = x, y$) at this node with BEM
7. Compute $\mathbf{v}_k(p)$ ($k = 0, 1, 2, 3; p \in \Omega$) with one-to-one mapping between vectors and crosses
8. End of iteration on all nodes
9. **#pragma omp barrier**

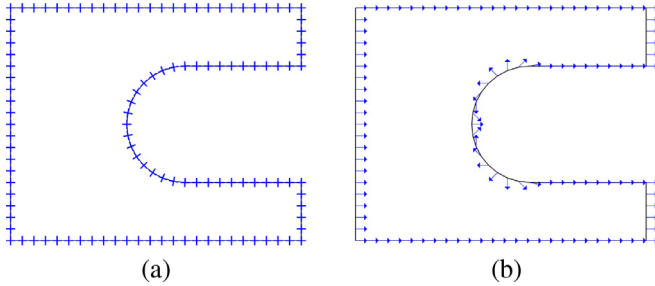


Fig. 3. Configuration for the computation of boundary values. (a) The crosses on the boundary. (b) The vectors on the boundary.

where S , T and P are the segment set, triangle set and node set, respectively. Because our aim is to propagate the vectors defined on the boundary, only Dirichlet boundary conditions are imposed on the boundary. When the constant elements are used, each element is assigned with a vector, i.e., the value of u_i ($i = x, y$) in Eq. (3) is given. However, the value of $\partial u_i/\partial n$ ($i = x, y$) is unknown. In order to solve $\partial u_i/\partial n$ ($i = x, y$), an equation in the form of Eq. (3) is formulated for each element e_i , and finally the linear system shown in Eq. (7) is formulated and solved.

Regarding to the initial vectors, they are determined by the desired crosses prescribed on the boundary of the problem domain via the mapping defined in Section 3.1. In order to get a boundary aligned quadrilateral mesh near the boundary, the crosses should also be set aligned with the boundary, i.e., two components of a cross is parallel to the boundary. And the norms of the initial vectors are set to the same positive value. Fig. 3a presents the desired crosses prescribed on the boundary of a 2D geometric model, and Fig. 3b presents the corresponding initial vectors. For case of corners featuring small angles (generally less than $\frac{\pi}{4}$), we use a similar approach in [11].

With the initial vector u_i ($i = x, y$) and the computed flux value $\partial u_i/\partial n$ ($i = x, y$) for each boundary element, we can use Eq. (8) to compute the solution of $u_i(p)$ ($i = x, y$) at any arbitrary point p located inside the computational domain. In this study, the vector at each triangular background mesh nodes is computed (see Fig. 4a). Accordingly, the cross at each node is computed via the mapping between a vector and a cross defined in Eq. (1) (see Fig. 4b).

Note that each component of the normal vector $u_i(p)$ ($i = x, y$) is governed by a Laplacian equation (see Eq. (2)). To accelerate the solving process of boundary integral equations in BEM, i.e., solution of boundary values and solution of inner vectors, the fast multipole method is adopted in this study [17–20]. The computational time for computing vectors at inner nodes is proportional to the number of inner nodes which may be very large for some domains. To accelerate this process, the parallelisation of the solution of inner vectors is implemented on a shared-memory computer with the parallel programming tool OpenMP.

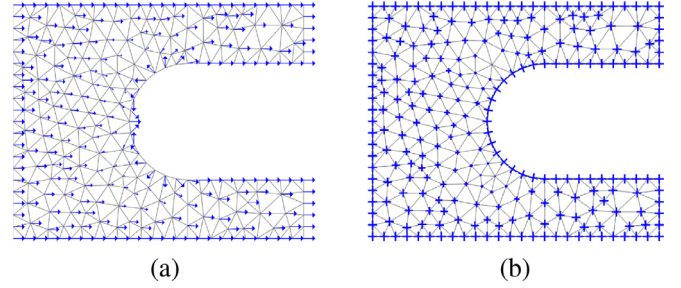


Fig. 4. Solution in the domain interior. (a) The vector-field. (b) The cross-field.

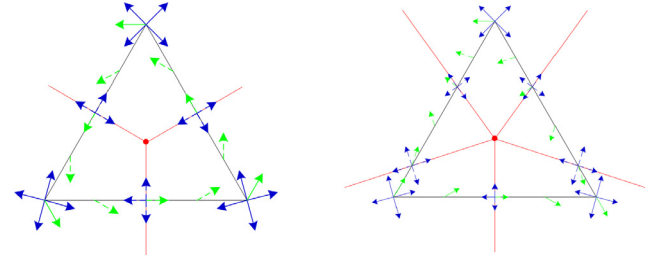


Fig. 5. Singular points located in triangles: a valence-3 singular point (left); a valence-5 singular point (right).

4.2. Domain decomposition

The generated vector-field and cross-field in the previous section represent the inner geometric information of the domain to some extent. And we can use them to decompose the domain into block-structured subdomains by analyzing the topology of these two fields. In this step, singular points are first identified by analyzing the vector-field, and then the streamlines that emanate from these singular points are extracted by analyzing the cross-field. After the streamlines are computed, to avoid some numerical issues, streamline simplifications are needed in order to get the final domain partitioning result.

4.2.1. Streamline extraction

The notions of singular points and streamlines are well demonstrated in [11,16], and they are briefly reviewed here for the sake of the completeness of this study:

1. **Singular point.** Given a vector-field $\mathbf{v}(x, y)$ over a domain Ω and a point $p \in \Omega$, we say p is a singular point when it satisfies $\mathbf{v}(p) = 0$.
2. **Streamline.** Given a cross-field $\mathbf{v}_k(p)$ ($k = 0, 1, 2, 3$) over a domain Ω , let $\gamma(s) \in \Omega$ ($s \in [0, 1]$) be a parametric curve and $\partial\gamma(s)$ be its derivative. We say $\gamma(s)$ is a streamline of $\mathbf{v}_k(p)$ when it satisfies $\forall s \in [0, 1], \exists \partial\gamma(s) \times \mathbf{v}_i(\gamma(s)) = 0$.

In this study, the vector-field and the cross-field are defined discretely on the nodes of the background triangular mesh, and the vector and cross at any other point are computed by piecewise linear interpolation. Then it is easy to identify the positions of points with zero vectors, and these points are regarded as singular points. Note that, a singular point may lie on the nodes, in the interior, or on the edges of a triangle.

The number of streamlines (refers to valence hereafter) coinciding at a singular point may vary, and it depends on the surrounding vectors computed on the background mesh of a singular point. As introduced in [11], the valence of a singular point is equal to either 3 or 5. Fig. 5 presents two singular points with different valences. After identifying the singular points and their valences, the streamlines are computed as the lines to partition the domain. For each singular point, compute all the streamlines emanating from it and trace them over the cross-field by visiting a series of triangles of the background mesh. Numerically,

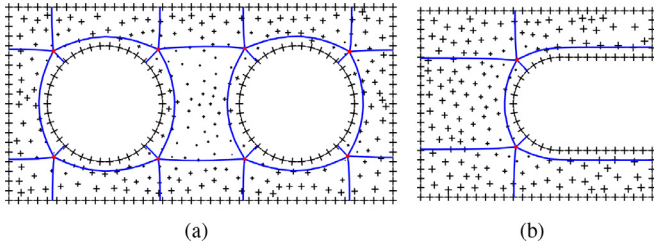


Fig. 6. The domain partitioning results for two 2D geometric models shown in previous sections.

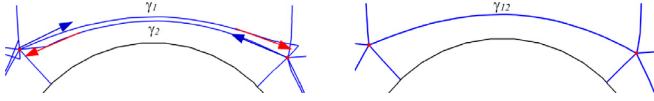


Fig. 7. Demonstration of the simplifications of streamlines: the initial streamlines (left); the simplified streamlines (right).

we use the method proposed in [11] to compute the streamlines. This method uses a 2nd-order Runge-Kutta scheme to integrate the streamline over a background triangular element, with an adaptive integration step based on the shape and size of the element. For each streamline, the tracing process is terminated when it reaches either a triangle containing another singular point or the boundary of the problem domain. Note that we need to interpolate the crosses and vectors at points inside triangular elements or on their edges during the integration of streamlines. In this study, we use vectors with varying scales that directly computed by BEM for such interpolation purpose, which is quite different from that using normalized vectors in [11]. Fig. 6 presents the domain partitioning results for two 2D geometric models shown in the previous sections.

4.2.2. Streamline simplification

As the streamlines are computed with the Runge-Kutta scheme based on the discretized cross-field defined on a background mesh, some redundant streamlines may be found due to the numerical issues. To get the final domain decomposition result, some simplifications on the initial streamlines are needed (see Fig. 7). The main operation in the process of streamline simplification is the merging of different streamlines into a single streamline. Take the streamline simplifications in Fig. 7 for example, the streamlines γ_1 and γ_2 (left) are merged into the streamline γ_{12} (right). To this end, given a singular point, a flag f is attached to each streamline connected to this point, specifying whether the streamline emanates from this point ($f = out$) or reaches this point from other points ($f = in$). The blue and red arrows in Fig. 7 represent a streamline emanates from and reaches a point, respectively. Then, this flag is used to identify which group of streamlines should be merged with each other. From the generation of streamlines, we know that the streamlines emanating from a singular point (with $f = out$) represent the ideal streamlines connecting to this point. Thus, for each singular point, the streamlines with $f = out$ are taken as reference streamlines, and the streamlines with $f = in$ are identified to be merged with appropriate reference streamlines. For example, given a singular point, for each streamline (denoted as γ) connected to this point with $f = in$, a streamline γ_i ($i = 1, 2, 3$ or $i = 1, \dots, 5$, depending on its valence) with $f = out$ is identified as the streamline that γ should be merged with. The criterion that determines whether the streamline γ should be merged with the streamline γ_i is to check the angle θ between these two lines:

$$\begin{cases} 120^\circ \leq \theta \leq 240^\circ, & \text{for a 3-valence singular point} \\ 144^\circ \leq \theta \leq 216^\circ, & \text{for a 5-valence singular point} \end{cases} \quad (9)$$

If the above criterion is satisfied, then γ should be merged with a streamline γ_i .

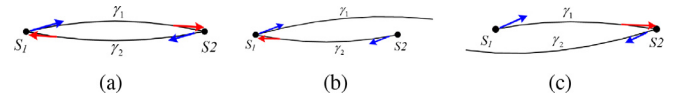


Fig. 8. Three different cases for streamlines need to be simplified between two singular points: (a) γ_1 emanating from S_1 reaches S_2 , and γ_2 emanating from S_2 reaches S_1 ; (b) γ_1 emanates from S_1 but doesn't reach S_2 , and γ_2 emanating from S_2 reaches S_1 ; (c) γ_1 emanating from S_1 reaches S_2 , and γ_2 emanates from S_2 but doesn't reach S_1 .

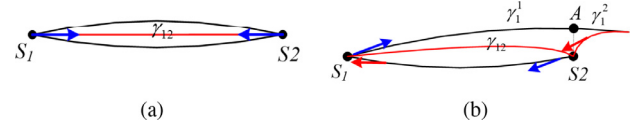


Fig. 9. The streamline simplification results for two cases listed in Fig. 8: (a) the first case shown in Fig. 8a; (b) and the second case shown in Fig. 9b.

In this study, the streamline simplifications mainly happen between two singular points. Given two singular points S_1 and S_2 , and suppose streamlines γ_1 and γ_2 are two candidate streamlines that need to be simplified between these two points based on the merging criterion. There are three topological relationships between these two streamlines and these two singular points: (1) γ_1 emanating from S_1 reaches S_2 , and γ_2 emanating from S_2 reaches S_1 ; (2) γ_1 emanates from S_1 but doesn't reach S_2 , and γ_2 emanating from S_2 reaches S_1 ; (3) γ_1 emanating from S_1 reaches S_2 and γ_2 emanates from S_2 but doesn't reach S_1 (see Fig. 8). The simplification for the first case is very straightforward; a new streamline γ_{12} is obtained by interpolation between γ_1 and γ_2 (see the red line in Fig. 9a):

$$\gamma_{12}(s) = (1-s)\gamma_1(s) + s\gamma_2(s), \quad s \in [0, 1]. \quad (10)$$

The simplifications for the second and third cases are similar, and here we only discuss the second case. First, a point A is generated on γ_1 by projecting S_2 onto γ_1 , which splits γ_1 into two parts γ_1^1 and γ_1^2 . Then, a new streamline γ_{12} is obtained by interpolation between γ_1^1 and γ_2 with Eq. (9) (see the red line between S_1 and S_2 in Fig. 9b). At the same time, the flag f of γ_1^2 at S_2 is updated as $f = in$ (see red line on the right of S_2 in Fig. 9b), and its simplification is done at the singular point S_2 just as the streamlines reaches S_1 .

Based on the above discussion, Algorithm 2 is formulated to simplify the initial streamlines generated in Section 4.2.1. A stack is created to store all the singular points. Each time a point being popped out, the streamlines connecting to it with $f = in$ is processed to merge with an appropriate streamline connecting to this point with $f = out$. And when the second case shown in Fig. 8b happens, the singular point S_2 should be pushed into the stack if it is not already in the stack.

Note that sometimes the initial streamlines may circulate around some curved inner boundary and never get closed due to the numerical integration error. In this study, these streamlines can be identified by checking whether a streamline passes through the same background mesh element. When a circular streamline is identified, the singular point where the streamline emanates from (regarded as the source point of this streamline) is projected to the streamline on a new point, and this point is then merged with the source singular point by adjusting the position of this streamline. Finally, a closed streamline will be generated.

5. Quadrilateral mesh generation

Once the problem domain is partitioned into several four-sided subdomains, it is ready to generate quadrilateral meshes for these subdomains. As all the subdomains are four-sided and there are no further singular points in these subdomains, structured quadrilateral meshing algorithms can be used to generate a structured quadrilateral mesh in each subdomain. In this study, the transfinite interpolation method

Algorithm 2 Streamline simplification

1. Input: singular point S_k ($k=0,1,\dots,m$) and initial streamlines γ_k ($k=0,1,\dots,m$)
2. Push all the S_k ($k=0,1,\dots,m$) into a stack $StkPt$.
3. While $StkPt$ is not empty
4. Get a singular point S_c from $StkPt$: $S_c = StkPt.pop()$
5. Get all the streamlines connecting to S_c , and divide them into two sets: $\gamma_{out} = \{\gamma_i | \gamma_i: f = out \text{ at } S_c; i = 1, 2, 3 \text{ or } i = 1, \dots, 5\}$ and $\gamma_{in} = \{\gamma_j | \gamma_j: f = in \text{ at } S_c; j = 1, \dots, k\}$
6. For each γ_j in γ_{in}
7. Find a streamline γ_i from γ_{out} that should be merged with γ_j , based on the criterion shown in Eq. (9)
8. Determine the streamline simplification case (C_{Sim}) shown in Figure 8.
9. Switch (C_{Sim})
10. Case 1 (see Figure 8a):
11. Merge γ_i and γ_j with Eq. (10), see Figure 9a
12. break;
13. Case 2 (see Figure 8b):
14. Split γ_i into two parts: γ_i^1 and γ_i^2 ;
15. Merge γ_i^1 and γ_j with Eq. (10), see Figure 9b;
16. Set $\gamma_i^2: f = in$ at S_j where γ_j emanates from, and push S_j into $StkPt$ if it is not in $StkPt$.
17. break;
18. End switch
19. End for
20. End while

[22] is adopted, which works on the parametric space of each subdomain and is quite robust and efficient. Like many other mesh generators, the meshing procedure follows a bottom-up procedure, i.e., it first meshes the curves and then meshes the subdomains individually. After the discretization of each subdomain, the mesh of the whole domain is obtained by combining all the meshes of the subdomains.

Note that before meshing the curves, it is better to first determine the intervals of each side of all the subdomains. There are two requirements that we need to meet. Firstly, to ensure the conformity of the mesh at the common sides between different subdomains, the discretization of these sides should be the same. Secondly, in order to ensure the generation of structured quadrilateral meshes on each subdomain, the number of intervals on the opposing sides of a subdomain should be equal. As in the implementation, the curves are discretized first and each curve is discretized only once, the first requirement will be met naturally. In order to satisfy the second requirement, a linear optimization problem could be formulated to determine the final number of intervals for each curve.

Suppose there are m curves and n subdomains in the domain decomposition result, and let c_i^0 ($i=0,1,\dots,m-1$) be the initial number of intervals adapted to a prescribed mesh sizing function, and c_i ($i=0,1,\dots,m-1$) be the final number of intervals, a minimization problem can be formulated as follows:

$$\begin{aligned} \min & \sum_{i=0}^{m-1} |c_i - c_i^0| \\ \text{s.t.} & \begin{cases} c_j^l = c_j^r, \\ c_j^b = c_j^t, \end{cases} (j=0,1,\dots,n-1). \end{aligned} \quad (11)$$

Here, c_j^l , c_j^r , c_j^b and c_j^t are local indices of curves in the j th subdomain, representing the two pair of sides of a subdomain.

Normally, the number of intervals of each curve is only allowed to increase when in solving Eq. (11) in case of downgrading the resolution of the desired mesh. Thus, the objective function in Eq. (11) can be changed to a linear function which leads to a linear optimization problem that can be efficiently solved by the open source library

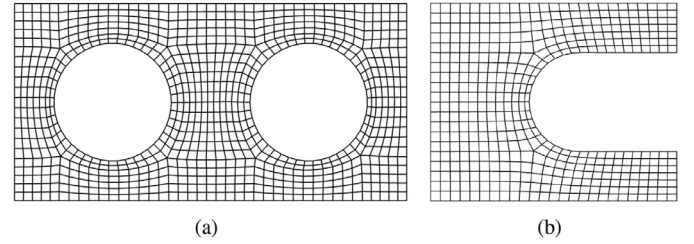


Fig. 10. The generated quadrilateral meshes of the two geometric models used in previous sections.

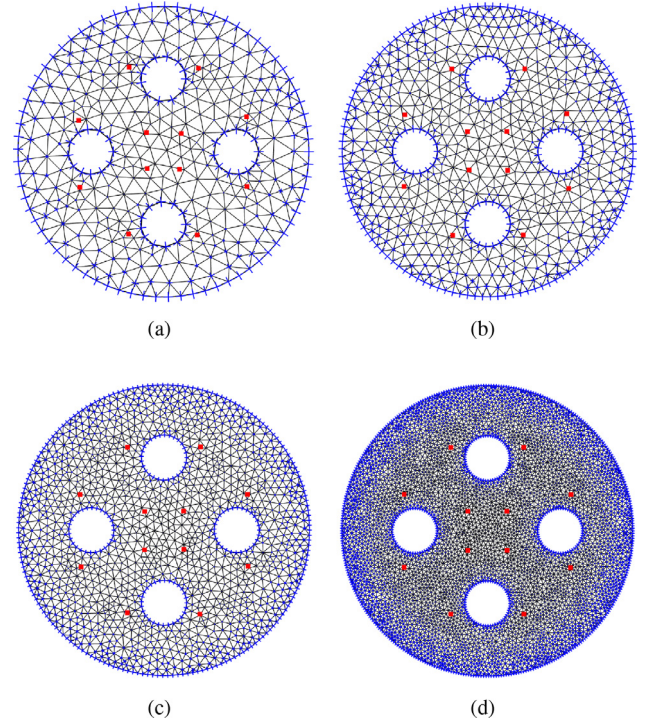


Fig. 11. cross-fields and singular points computed on background meshes with different magnitudes: (a) 657 triangles; (b) 1,160 triangles; (c) 2,519 triangles; (d) 9,990 triangles.

lpsolve [23]. Fig. 10 presents the generated quadrilateral meshes of the two geometric models used in previous sections.

6. Numerical experiments

A collection of test cases are used to evaluate the effectiveness of the algorithm proposed in this study, and they were all conducted on a desktop computer (CPU: Intel Core i3-6100 3.7 GHz; memory: 4 GB). Each test starts with a 2D geometric model, and automatically outputs a domain partitioning result and a quadrilateral mesh. During the whole process, only two user parameters are needed: the mesh sizes for the background mesh and the final quadrilateral mesh, respectively. The background meshes of these models are generated using an in-house surface mesher based on the Delaunay method.

To demonstrate the accuracy of the proposed method in solving the governing PDEs for cross-field, we use a four-hole model to compare the results obtained in this study with the results presented in [11]. Fig. 11 presents the distribution of cross-fields computed on different scales of background meshes with the proposed method. The mesh in Fig. 11a contains 657 triangle elements, while the remaining three sets of mesh contain roughly 2, 4 and 16 times more triangle elements. The singular points of these cross-fields are also identified (see the red dots in Fig. 11).

Table 1
The main performance statistics of the tests conducted in this study.

Test Model	#Singular points	#Subdomains	#Elements	#Node	Min angle	Min scaled Jacobian	Time(s)
Example-1 (Fig. 15)	12	75	1,284	1,372	46.6	0.72	0.37
Example-2 (Fig. 16)	12	101	2,506	2,654	51.7	0.77	0.69
Example-3 (Fig. 17)	19	217	10,400	10,854	28.1	0.37	3.05
Example-4 (Fig. 18)	4	10	153	171	75.5	0.85	0.09
Example-5 (Fig. 19)	4	16	268	318	61.0	0.80	0.21
Example-6 (Fig. 21)	6	65	63,911	64,675	24.9	0.29	0.60
Example-6* (Fig. 22)	6	65	216,294	217,496	23.4	0.24	6.40

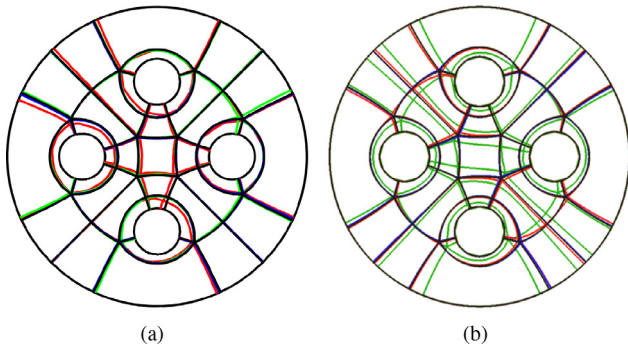


Fig. 12. Schematic diagram of sub-regions generated by the two methods based on different background mesh size: (a) Our method; (b) Kowalski's method.

Due to the symmetry of this model, the positions of singular points computed by the proposed method are almost symmetric in each background mesh. To compare the effects of different scales of background mesh on the solution of cross-fields, we present the domain partitioning results based on the computed cross-fields together in Fig. 12a. The green, red, blue, and black streamlines in Fig. 12a correspond to the result of the four sets of background mesh in Fig. 11, respectively. It can be seen that the streamlines with different colors almost overlap each other, which means the cross-fields computed with different scales of background mesh have comparable accuracy. This fact is proved further by the final quadrilateral meshes (see Fig. 13a) based on the inputs of the coarsest and the finest background mesh. For comparison, the results presented in [11] are also listed here. Fig. 12b presents the domain partitioning results (in green, red, blue and black color lines, respectively) based on different scales of background mesh, ranging from 860 triangle elements, to 4 times, 16 times and 64 times more elements. It can be seen that the results based on different scales of background mesh have big differences from each other. In particular, the final quadrilateral meshes (see Fig. 13b) based on the inputs of the coarsest and finest background mesh are quite different from each other.

Table 1 presents the performance statistics of another seven tests conducted in this study, including the number of singular points, the number of subdomains, the number of generated quadrilateral elements and nodes, the minimum interior angle of the mesh elements, the minimum scaled Jacobian quality measure of the mesh elements, and the time cost of the whole procedure. It can be seen that the proposed method runs very fast in terms of the speed of generating quadrilateral mesh which ranges from 1,276 elements (Example-5) to 106,518 elements (Example-6) per second. In addition, Table 2 presents the breakdown of the timing data of the whole procedure, i.e., solution of cross-field, domain partitioning and mesh generation. Note that the time of generating background mesh is included in the step of the solution of cross-field. Among all these steps, the step of solving cross-field consumes most of the time, i.e., this step costs 78% of the overall time for the Example-6 test model, and the percentage for the Example-2 model is even larger, 97%.

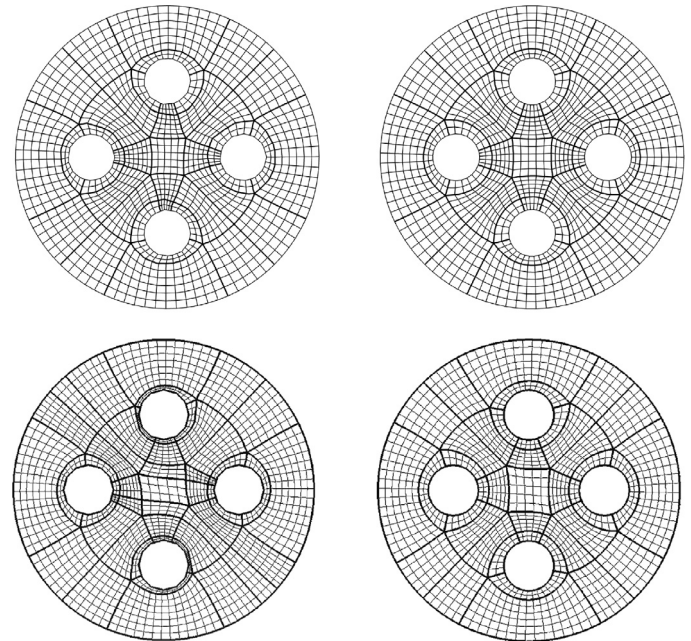


Fig. 13. Comparison between the proposed approach and the method proposed by Kowalski et al: (a) Our method; (b) Kowalski's method.

Table 2
Breakdown of the timing data of the whole procedure (seconds)

Model name	Solution of cross-field	Domain partitioning	Mesh generation
Example-1 (Fig. 15)	0.35	0.01	0.01
Example-2 (Fig. 16)	0.67	0.01	0.01
Example-3 (Fig. 17)	2.87	0.16	0.02
Example-4 (Fig. 18)	0.08	0.007	0.001
Example-5 (Fig. 19)	0.17	0.04	0.001
Example-6 (Fig. 21)	0.47	0.04	0.09
Example-6* (Fig. 22)	6.00	0.10	0.30

To evaluate the effectiveness of the proposed domain partitioning method, the quality of the generated quadrilateral elements is computed. The distributions of the minimum interior angle and the scaled Jacobian are used as the quality measures in this study, and they are presented in Fig. 14 for different meshes. As can be seen, most of the quadrilateral elements have quite good quality. For the minimum angle quality measure, the ratios of the minimum angles larger than 60 for these models are 99.5%, 99.6%, 86.8%, 100%, 99.9%, 98.6% and 98.3%, respectively, while the ratios of the minimum angles lower than 48 are 0.08%, 0.0%, 3.5%, 0.0%, 0.0%, 0.34% and 0.07%, respectively. For the scale Jacobian quality measure, the ratios of the scale Jacobian larger than 0.7 for these models are 100%, 100%, 97.7%, 100%, 100%, 99.8% and 99.9%, respectively, while the ratios of the scale Jacobian lower than 0.4 are 0.0%, 0.0%, 0.03%, 0.0%, 0.0%, 0.005% and 0.001%, respectively.

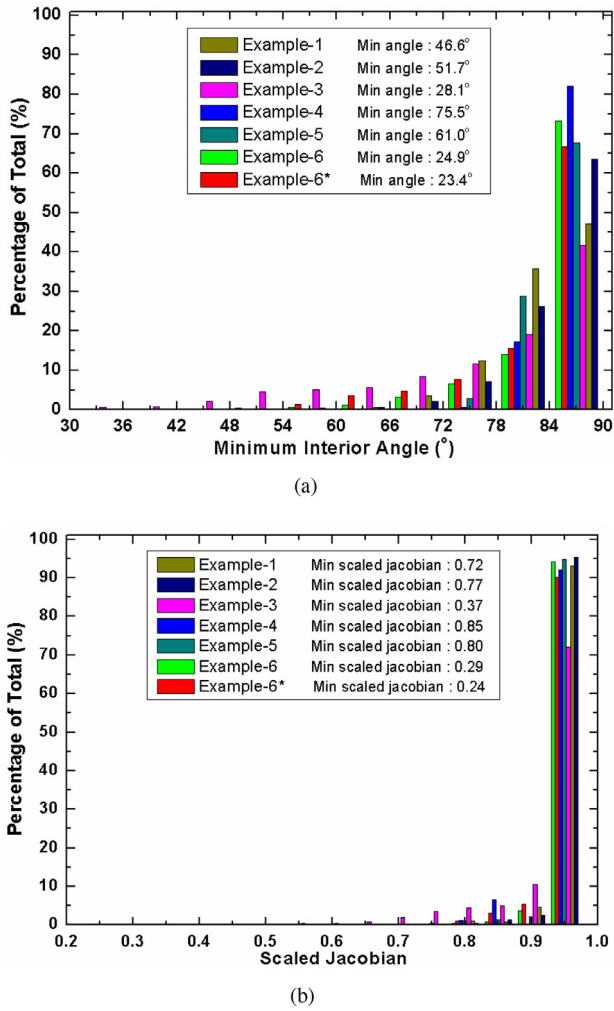


Fig. 14. The quality of the generated quadrilateral meshes for all the test models: (a) the distribution of minimum interior angle measure; (2) the distribution of scaled Jacobian quality measure.

Fig. 15 presents the domain partitioning process for the Example-1 model. Firstly, the boundary is discretized into line segments and at the same time a background mesh is generated in the interior region (Fig. 15a). Then, the boundary aligned initial crosses are defined on the boundary (Fig. 15b), and the corresponding reference vectors are computed on the boundary (Fig. 15c) which are propagated into the interior region smoothly (Fig. 15e). After that, the cross-field over the domain is computed by the mapping between vectors and crosses. Finally, the domain is partitioned into many four-sided subdomains by analyzing the vector-field and cross-field (Fig. 15g), and each subdomain is meshed with structured quadrilateral mesh (Fig. 15h). In this test, there are 12 singular points existing in the interior region and the streamlines emanating from them partition the domain into 75 subdomains.

In Fig. 16, we show how the domain partitioning changes when the geometry of the domain is slightly modified. In this case, a quadrilateral hole is added in the model presented in Fig. 15 (see Fig. 16a). Fig. 16b presents the generated cross-field, and the corresponding domain partitioning result is shown in Fig. 16c. Because the added quadrilateral hole does not introduce new singular points, the singular topology of this model remains unchanged, i.e., the number of singular points is still 12. However, the positions of some of these singular points slightly changes, compared with the result shown in Fig. 15. Besides, the number of generated subdomains increases from 75 to 101.

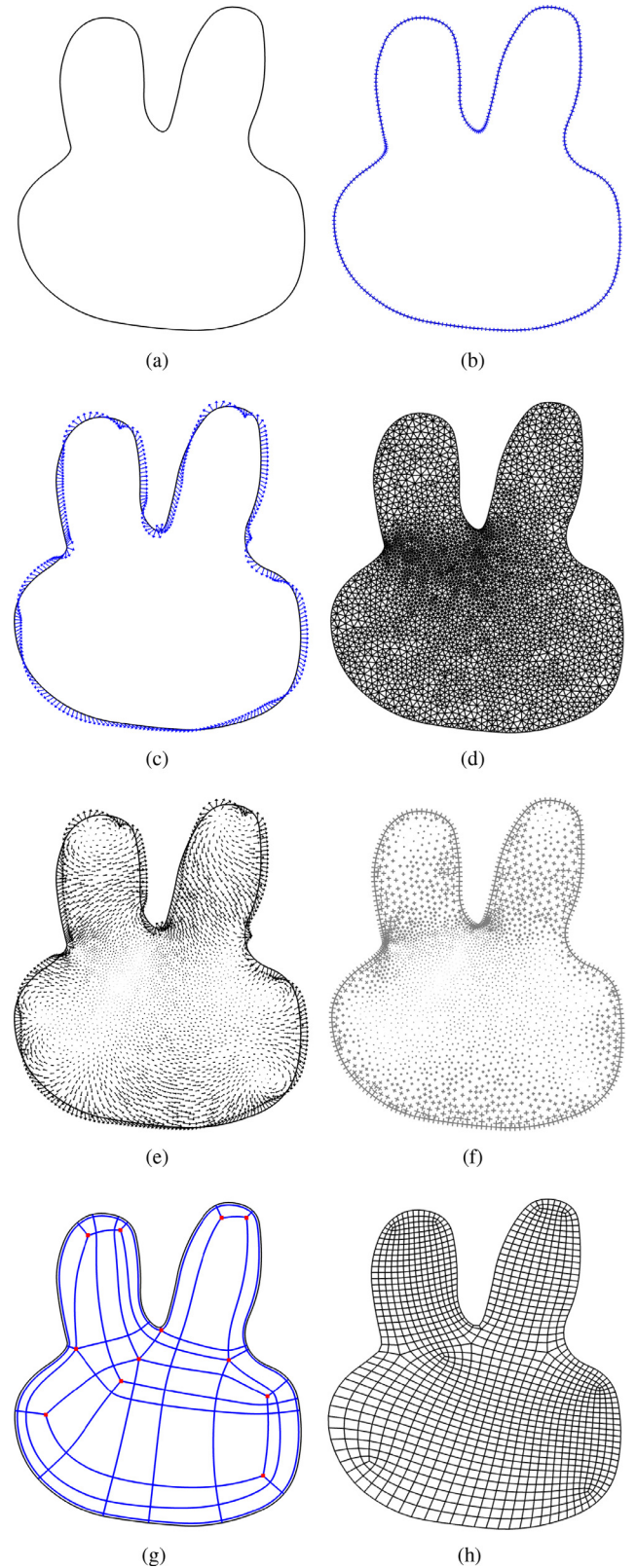


Fig. 15. Example-1. (a) The geometry. (b) and (c) The initial boundary crosses and vectors. (d) The background mesh. (e) The vector-field. (f) The cross-field. (g) The four-sided subdomains. (h) The block-structured quadrilateral mesh.

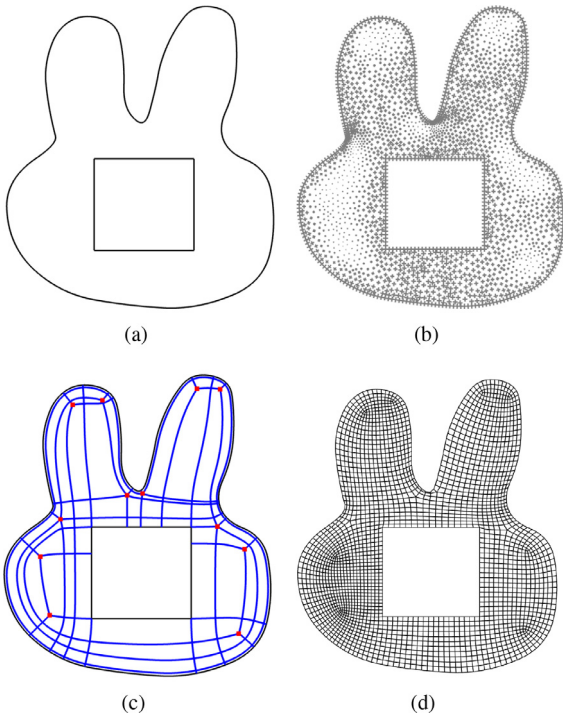


Fig. 16. Example-2. (a) The geometry. (b) The cross-field. (c) The four-sided subdomains. (d) The block-structured quadrilateral mesh.

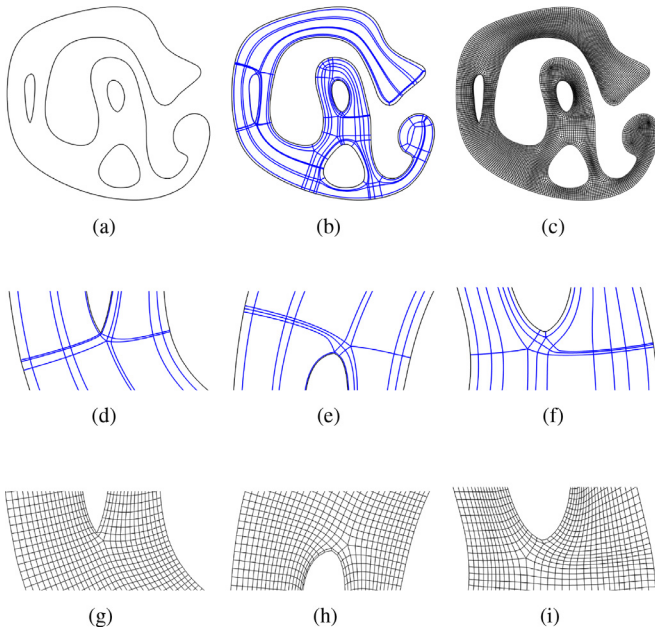


Fig. 17. Example-3. (a) The geometry. (b) The four-sided subdomains. (c) The block-structured quadrilateral mesh. (d)-(f) Close-up views of (b) at region near the holes. (g)-(i) Close-up views of (c) at regions near the holes.

To demonstrate the proposed method for complex configurations, we choose to partition the domain and generate a quadrilateral mesh for the Example-3 model shown in Fig. 17. This model is highly stretched and contains three irregular holes in its interior region (see Fig. 17a), which poses a great challenge to the domain partitioning method. As a result, 19 singular points are identified (including 6 valence-3 and 13 valence-5 singular points) in the interior region of the model and 217 subdomains are obtained (see Fig. 17b). The close-up views of the streamlines near the holes are presented in Fig. 17d-f, and as can be seen,

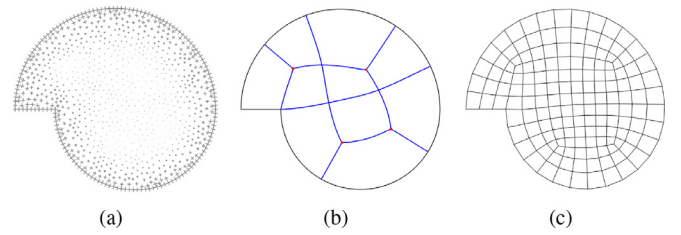


Fig. 18. Example-4. (a) The cross-field. (c) The four-sided subdomains. (d) The block-structured quadrilateral mesh.

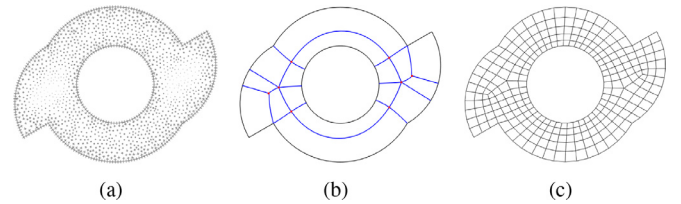


Fig. 19. Example-5. (a) The cross-field. (c) The four-sided subdomains. (d) The block-structured quadrilateral mesh.

many thin subdomains are generated in the result with two streamlines almost blend together. Normally, these streamlines are very likely to intersect with each other when other numerical methods are used to solve the cross-field. However, these potential intersections are avoided thanks to high accuracy of the solution of the cross-field obtained by BEM adopted in this study. Finally, the block-structured quadrilateral meshes are obtained (see Fig. 17c), and the mesh details near the holes are presented in the close-up views shown in Fig. 17g-i.

To demonstrate the ability of the proposed method in maintaining the symmetrical characteristic of the given geometric model, two deliberately built test cases (the Example-4 model and the Example-5 model, see Figs. 18–19) are chosen here. The Example-4 model is built by splitting the boundary of a circle at an arbitrary node and moving one end along the radial direction to a further position. In this way, the symmetrical characteristic of the circle is destroyed, and thus an unsymmetrical partitioning result is obtained. As a result, 4 valence-3 singular points are identified and 10 subdomains are obtained. Compared with a circle, the number of singular points does not change, but the number of subdomains increases by 5. Fig. 18 presents the cross-field, domain partitioning result and the generated quadrilateral mesh. Meanwhile, if a geometric model is symmetrical, a symmetrical domain partitioning result can also be obtained by the proposed method. For example, Fig. 19 presents the cross-field, domain partitioning result and the generated quadrilateral mesh for the Example-5 model which is a central symmetric geometric model. As shown in Fig. 19, four singular points (2 valence-3 points and 2 valence-5 points) are identified in the interior region and their positions are central symmetric. As a result, the partitions (including 16 subdomains) are also centrally symmetric (see Fig. 19b).

Note that, maintaining of the symmetrical characteristic in the partitions is not trivial. A symmetrical characteristic in geometry often leads to symmetrical physical features in the domain, and a symmetrical mesh would help to capture such features in the numerical simulation. Nonetheless, such an ability is not easy to achieve with other methods. For example, the parameterization-based method proposed by Bommers [7] creates degenerate elements for the Example-5 model (see Fig. 20a), and the Morse-Parameterization based method proposed by Fang [1] generates an unsymmetrical mesh result with one singular point located on the boundary and other three singular points located in the interior region [1] (see Fig. 20b).

To demonstrate the application of the proposed method to real geometric models, we choose to generate quadrilateral mesh for the

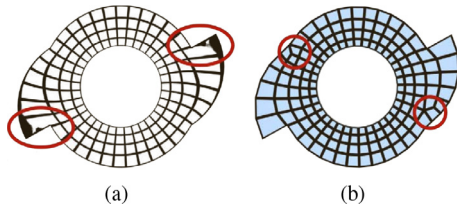


Fig. 20. Quadrilateral meshes from (a) [7] and (b) [11].

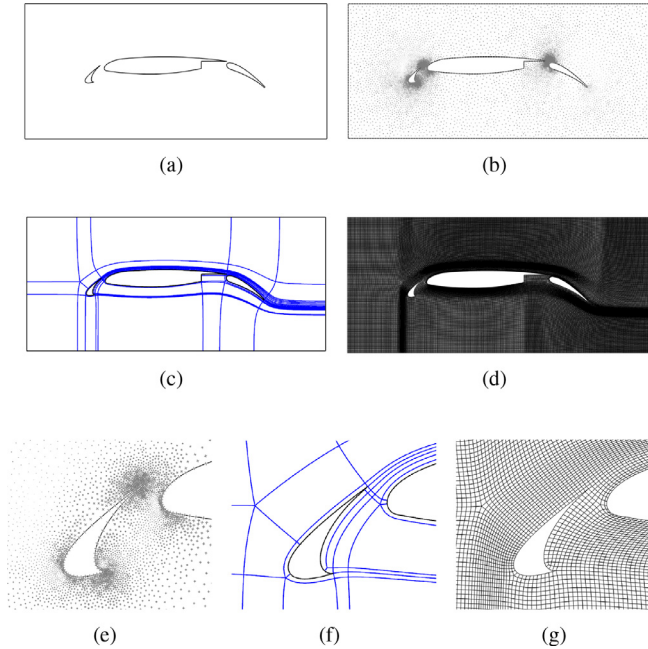


Fig. 21. Block-structured quadrilateral meshing of the Example-6 model (an airfoil configuration composed of three components: slat, main element and flap). (a) The geometry. (b) The cross-field. (c) The four-sided subdomains. (d) The mesh. (e)-(g) Close-up views of (b)-(d) at a small region around slat.

Example-6 model. This model is the 30P/30N three-element high-lift airfoil which is composed of a slat, a main element and a flap [24,25]. Many complex geometrical features are involved in this model. For example, narrow gaps exist between the slat and the main element, the flat and the main element, and cusps are observed on the entire three elements (see Fig. 21a). These complex geometrical features complicate the problem of quadrilateral mesh generation around those regions. In the partitioning process, the boundary of the model is configured with boundary aligned crosses. Fig. 21b presents the computed cross-field in the interior region and a close-up view around the slat is presented in Fig. 21e. Based on the cross-field, 6 singular points (including 1 valence-3 and 5 valence-5 points) are identified and the domain is partitioned into 65 four-sided subdomains (see Fig. 21c and the close-up view around the slat in Fig. 21f). Finally, block-structured quadrilateral meshes are obtained by meshing each subdomain individually (see Fig. 21d and g). Because the boundary aligned crosses are set as the boundary conditions when solving the cross-field, boundary aligned quadrilateral elements can be finally generated in the vicinity of boundary; this property is crucial in increasing the solution accuracy of CFD simulations [21].

To evaluate the validity of the generated quadrilateral mesh, a viscous flow simulation was conducted around the 30P/30N three-element high-lift airfoil model. In order to achieve that, a slight modification was made to the model by adjusting the size of the bounding box (see Fig. 22a, referred to as Example-6* hereafter). The mesh generation process is the same as Example-6, and the mesh statistics are shown in Table 1. During the simulation, the left side and right side of the bound-

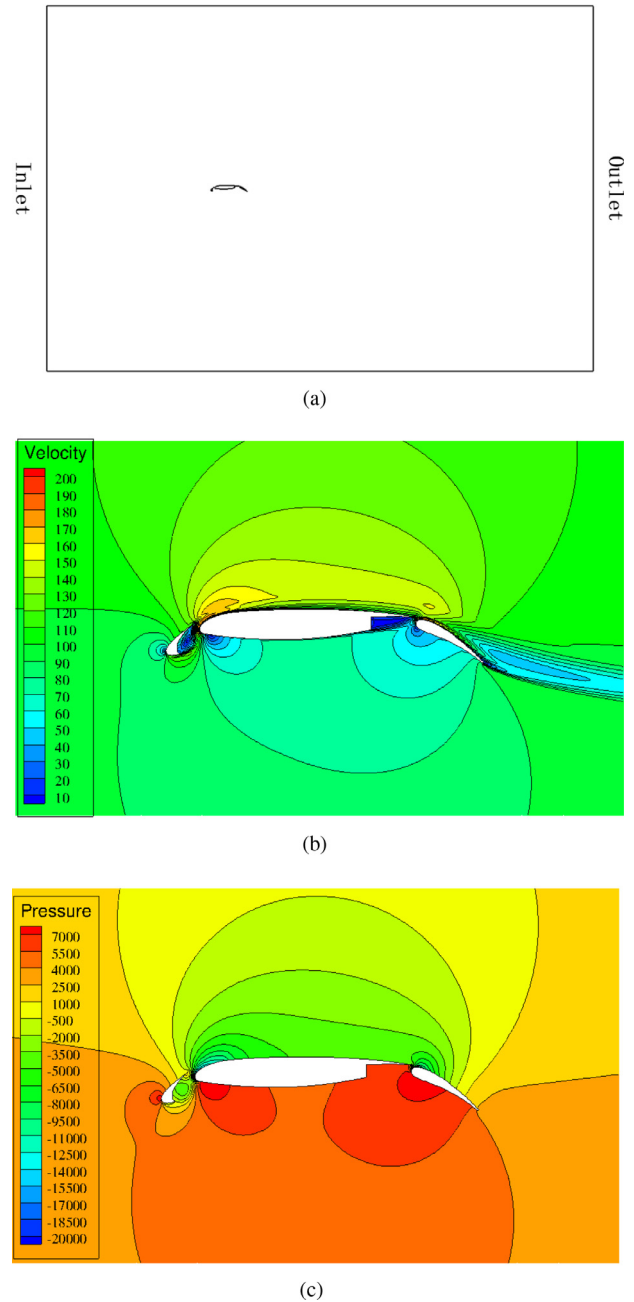


Fig. 22. Viscous flow simulation for the Example-6* model. (a) The geometry. (b) Contour plot of the computed velocity distributions (m/s). (c) Contour plot of the computed pressure distributions (Pa).

ing box are set as inlet and outlet boundary conditions, respectively. On other parts of the boundary no-slip boundary condition is imposed. The Mach number for this simulation was 2.9. Fig. 22b and c present the contour plots of the computed velocity and pressure distributions during the simulation. From the contour plots, we can see that the flow speed below the airfoil is slower than that above the airfoil (see Fig. 22b), while the pressure below the airfoil is larger than that above the airfoil (see Fig. 22c).

Finally, to test the robustness of the proposed method, more tests have been done with a variety of models. These models have different complex features, i.e., holes, sharp angles and reversing sharp angles. The domain partitioning results and final quadrilateral meshes of these models are presented in Fig. 23, and their performance statistics are presented in Table 3.

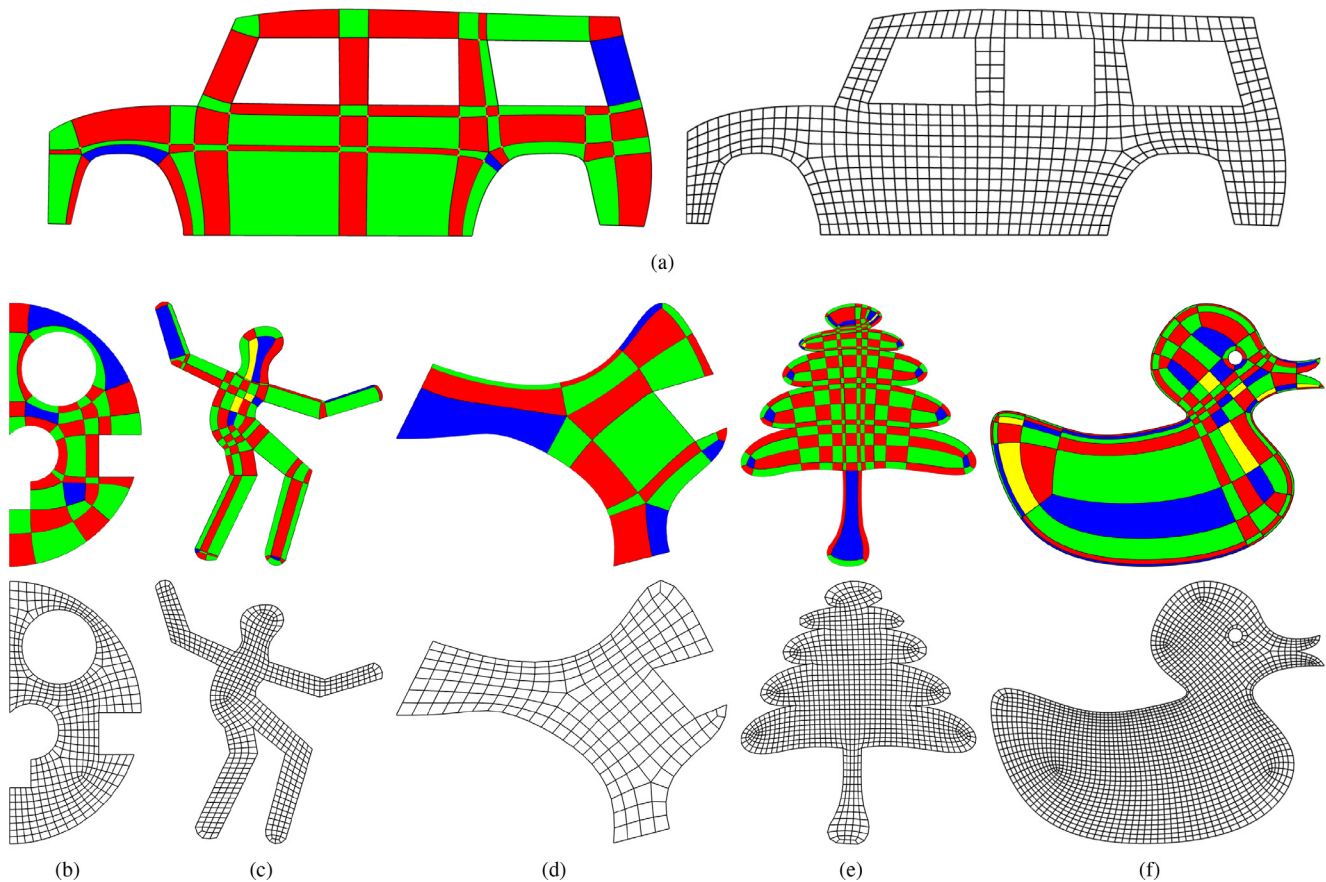


Fig. 23. Gallery. Collection of domain partitioning results and quadrilateral meshes for a variety of models: (a) Car; (b) Fish; (c) Person; (d) Axis; (e) Tree; (f) Duck.

Table 3

Performance statistics of the tests in the gallery.

Test model	#Singular points	#Subdomains	#Elements	#Node	Min angle	Min scaled Jacobian
Car	3	60	644	765	61.1	0.42
Fish	6	40	424	501	50.0	0.41
Person	8	82	578	694	48.4	0.31
Aix	4	22	210	251	52.4	0.33
Tree	22	237	1,059	1,155	43.0	0.44
Duck	14	135	2,097	2,205	41.5	0.24

7. Concluding remarks

A domain partitioning method is proposed in this paper for automatically partitioning any given 2D domain into regions suitable for quadrilateral meshing. It uses two Laplacian equations to control the propagation of crosses defined on the boundary into the interior domain, thus a smooth cross-field is obtained over the domain. By analysing this cross-field, singular points are identified and streamlines emanating from them are computed as lines to partition the domain. This method can generate four-sided subdomains which can be discretized with structured quadrilateral meshes, and finally a block-structured mesh is obtained. The Laplacian equations are solved with the boundary element method (BEM) which only needs a discretized boundary as the input and achieves a high accurate cross-field over the domain. Furthermore, the fast multi-pole method is used to accelerate the solution process of the boundary integral equations in BEM, and OpenMP is used to compute crosses on the background mesh nodes simultaneously. These two measures help to reduce the solution time for the cross-field substantially and therefore lead to a very efficient domain partitioning method.

Acknowledgements

The authors appreciate the joint support for this project by the [National Natural Science Foundation of China](#) (Grant Nos. [61802099](#), [61761136010](#), [61772163](#), and [U1630121](#)), Zhejiang Provincial Natural Science Foundation of China (Grant No. LY17F020016 and LR16F020002), Aeronautics Power Foundation (Grant No. 6141B09050373) and the NSFC-Zhejiang Joint Fund for the Integration of Industrialization and Informatization (Grant No. U1909210). The first author acknowledges the support from China Scholarship Council and the host of Professor Carl Ollivier-Gooch for his postdoctoral research at The University of British Columbia, Canada.

References

- [1] Fang X, Bao H, Tong Y, Desbrun M, Huang J. Quadrangulation through morse-parameterization hybridization. *ACM Trans Graph* 2018;37(4):92:1–92:15.
- [2] Kälberer F, Nieser M, Polthier K. Quadcover - surface parameterization using branched coverings. *Comput Graph Forum* 2007;26(3):375–84.
- [3] Staten ML, Owen SJ, Blacker TD. Unconstrained paving & plastering: a new idea for all hexahedral mesh generation. In: *Proceedings of the 14th international mesh-*

- ing roundtable, IMR 2005, September 11–14, 2005, San Diego, CA, USA; 2005. p. 399–416.
- [4] Owen SJ, Staten ML, Canann SA, Saigal S. Q-morph: an indirect approach to advancing front quad meshing. *Int J Numer Method Eng* 2015;44(9):1317–40.
 - [5] Bommes D, Lévy B, Pietroni N, Puppo E, Silva CT, Tarini M, et al. Quad-mesh generation and processing: a survey. *Comput Graph Forum* 2013;32(6):51–76.
 - [6] Campen M, Bommes D, Kobbelt L. Quantized global parametrization. *ACM Trans Graph* 2015;34(6):192:1–192:12.
 - [7] Bommes D, Zimmer H, Kobbelt L. Mixed-integer quadrangulation. *ACM Trans Graph* 2009;28(3):77:1–77:10.
 - [8] Ray N, Vallet B, Li W, Lévy B. N-symmetry direction field design. *ACM Trans Graph* 2008;27(2):10:1–10:13.
 - [9] Shepherd JF, Dewey MW, Woodbury AC, Benzley SE, Staten ML, Owen SJ. Adaptive mesh coarsening for quadrilateral and hexahedral meshes. *Finite Element Anal Des* 2010;46(1):17–32.
 - [10] Fogg H, Armstrong C, Robinson TT. Multi_block decomposition using cross-fields. *Proc Adapt Modell Simulat Lisbon* 2013:254–67.
 - [11] Kowalski N, Ledoux F, Frey P. Automatic domain partitioning for quadrilateral meshing with line constraints. *Eng Comput (Lond)* 2015;31(3):405–21.
 - [12] Dong S, Bremer P, Garland M, Pascucci V, Hart JC. Spectral surface quadrangulation. *ACM Trans Graph* 2006;25(3):1057–66.
 - [13] Huang J, Zhang M, Ma J, Liu X, Kobbelt L, Bao H. Spectral quadrangulation with orientation and alignment control. *ACM Trans Graph* 2008;27(5):147:1–147:9.
 - [14] Ling R, Huang J, Jüttler B, Sun F, Bao H, Wang W. Spectral quadrangulation with feature curve alignment and element size control. *ACM Trans Graph* 2014;34(1):11:1–11:11.
 - [15] Tam TKH, Armstrong CG. 2d finite element mesh generation by medial axis subdivision. *Adv Eng Softw Workstat* 1991;13(5–6):313–24.
 - [16] Palacios J, Zhang E. Rotational symmetry field design on surfaces. *ACM Trans Graph* 2007;26(3):55.
 - [17] Yijun L. Fast multipole boundary element method: theory and applications in engineering. Cambridge Univ Press Cambridge 2009;9(275):xviii + 235.
 - [18] Zhang J, Zhuang C, Qin X, Li G, Sheng X. Fmm-accelerated hybrid boundary node method for multi-domain problems. *Eng Anal Bound Element* 2010;34(5):433–9.
 - [19] Zhang J, Tanaka M, Endo M. The hybrid boundary node method accelerated by fast multipole expansion technique for 3d potential problems. *Int J Numer Method Eng* 2005;63(5):660–80.
 - [20] Zhang J, Tanaka M. Adaptive spatial decomposition in fast multipole method. *J Comput Phys* 2007;226(1):17–28.
 - [21] Yao Z, Xiao Z, Chen J, Zhang J. Novel methodology for viscous-layer meshing by the boundary element method. *Aiaa J* 2017;56(1):1–13.
 - [22] Faux ID, Pratt MJ. Computational Geometry for Design and Manufacture. New York, NY, USA: Halsted Press; 1979. ISBN 047026473X.
 - [23] Berkelaar M.. lpsolve: Interface to 'lp_solve' v. 5.5 to solve linear/integer programs2015;.
 - [24] Anderson WK, Bonhaus DL, Mcghee RJ, Walker BS. Navier-stokes computations and experimental comparisons for multielement airfoil configurations. *J Aircraft* 1993;32(6):1246–53.
 - [25] Murayama M, Nakakita K, Yamamoto K, Ura H, Ito Y, Choudhari MM. Experimental study on slat noise from 30p30n three-element high-lift airfoil at jaxa hard-wall lowspeed wind tunnel. In: *Aiaa/ceas Aeroacoustics Conference*; 2014.